

Module I: INTRODUCTION TO WEB TECHNOLOGIES

Section	Title	Page
1.1	Introduction to the Internet	1
1.1.1	History of Internet	1
1.1.2	Internet in the age of AI	3
1.2	Evolution of the World Wide Web (WWW)	4
1.3	Web Browsers and Web Servers	11
1.3.1	Web Browser	11
1.3.2	Web Server	12
1.4	Client–Server Architecture	15
1.4.1	Components of Client–Server Architecture	16
1.4.2	Working of Client–Server Architecture	18
1.4.3	Types of Client–Server Architecture	18
1.4.4	Advantages of Client–Server Architecture	19
1.4.5	Disadvantages of Client–Server Architecture	20
1.4.6	Applications of Client–Server Architecture	20
1.5	HTTP and HTTPS Protocols	21
1.5.1	Hypertext Transfer Protocol (HTTP)	21
1.5.2	Hypertext Transfer Protocol Secure (HTTPS)	22
1.5.3	HTTP Request Methods	23
1.5.4	Structure of HTTP Request	23
1.5.5	Structure of HTTP Response	24
1.5.6	Common HTTP Status Codes	25
1.5.7	Difference Between HTTP and HTTPS	25
1.5.8	Advantages of HTTPS	26
1.5.9	Applications of HTTP and HTTPS	26
1.5.10	HTTP vs HTTPS Communication Flow	27
1.6	URL, URI and Domain Name System (DNS) Basics	28
1.6.1	Uniform Resource Identifier (URI)	28
1.6.2	Uniform Resource Locator (URL)	29
1.6.3	Difference Between URI and URL	31
1.6.4	Domain Name System (DNS)	31
1.6.5	Components of DNS	32
1.6.6	DNS Resolution Process	33
1.6.7	DNS Hierarchy	33
1.6.8	Common DNS Record Types	34
1.6.9	Advantages of DNS	34
1.6.10	Difference Between URL, URI, and DNS	35
1.6.11	Real-World Example	35
1.7	HTML Basics and Document Structure	36
1.7.1	Features of HTML	36
1.7.2	Basic Structure of an HTML Document	36
1.7.3	Explanation of HTML Document Structure	37
1.7.4	HTML Document Tree	38
1.7.5	HTML Tags	38
1.7.6	HTML Elements	39
1.7.7	HTML Attributes	40
1.7.8	Basic HTML Tags	41
1.7.9	HTML Comments	42

1.7.10	HTML Document Workflow	42
1.7.11	HTML Document Structure Diagram	42
1.7.13	Limitations of HTML	43
1.8	Creating a Simple Web Page	44

CHAPTER 1

INTRODUCTION TO WEB TECHNOLOGIES

1.1 Introduction to the Internet

The Internet is a global network of interconnected computers and digital devices that communicate using standardized protocols, primarily the Transmission Control Protocol (TCP) and the Internet Protocol (IP). It enables users across the world to exchange information, communicate instantly, access services, and share resources regardless of geographical location.

Initially developed for research and defense purposes, the Internet has evolved into an essential part of everyday life. Today, it supports a wide range of applications including email, web browsing, cloud computing, online banking, e-commerce, social networking, video conferencing, digital entertainment, and online education. Businesses, governments, educational institutions, and individuals rely on the Internet for communication, collaboration, and information sharing.

The Internet functions through a vast infrastructure of servers, routers, switches, communication links, satellites, and fiber-optic networks. Millions of websites and web applications are hosted on servers connected to this infrastructure, allowing users to access information using web browsers.

The rapid growth of mobile devices, wireless communication technologies, and cloud computing has further expanded Internet accessibility. With the emergence of technologies such as the Internet of Things (IoT), Artificial Intelligence (AI), and 5G networks, the Internet continues to evolve into a more intelligent, interconnected, and data-driven ecosystem.

1.1.1 History of Internet

The history of the Internet spans several decades and represents one of the most significant technological achievements of the modern era.

The foundation of the Internet began during the late 1960s with the development of ARPANET, a research network funded by the Advanced Research Projects Agency. ARPANET was designed to enable communication between computers at different research

institutions while ensuring reliable data transmission even if parts of the network failed.

During the 1970s, researchers Vinton Cerf and Robert E. Kahn developed the TCP/IP protocol suite, which standardized communication between different computer networks. On January 1, 1983, TCP/IP became the standard protocol for ARPANET, marking the birth of the modern Internet.

The 1980s witnessed the expansion of networking to universities and research organizations worldwide. The introduction of the Domain Name System (DNS) made it easier for users to access websites using readable domain names instead of numerical IP addresses.

A major breakthrough occurred in 1989 when Tim Berners-Lee proposed the World Wide Web (WWW). The World Wide Web introduced Hypertext Transfer Protocol (HTTP), Hypertext Markup Language (HTML), and web browsers, making information accessible through linked web pages.

The 1990s saw the commercialization of the Internet. Internet Service Providers (ISPs) offered public Internet access, and businesses began establishing websites. Search engines such as Yahoo! and later Google transformed how users located information online. E-commerce platforms also emerged, enabling online shopping and digital transactions.

In the 2000s, broadband Internet replaced dial-up connections, significantly increasing Internet speed. Social media platforms, cloud computing, online streaming services, and smartphones revolutionized Internet usage by making digital services accessible anytime and anywhere.

The 2010s introduced widespread adoption of cloud computing, Internet of Things (IoT), big data analytics, and high-speed mobile Internet through 4G and later 5G technologies. These developments enabled real-time communication, smart devices, and large-scale digital transformation.

Today, the Internet serves billions of users worldwide and continues to evolve through advances in artificial intelligence, edge computing, quantum networking research, and next-generation communication technologies.

1.1.2 Internet in the age of AI

Artificial Intelligence (AI) is transforming the Internet from a platform that simply provides information into one that understands, predicts, and responds intelligently to user needs. Modern Internet services increasingly rely on AI algorithms to enhance user experience, automate tasks, improve security, and provide personalized services.

Search engines now use AI to understand user intent rather than relying solely on keyword matching. Recommendation systems on streaming platforms, e-commerce websites, and social media analyze user behavior to suggest relevant content, products, and services. AI-powered virtual assistants and chatbots provide instant customer support, answer questions, and automate routine interactions.

AI also plays a crucial role in cybersecurity by detecting malware, phishing attacks, network intrusions, and unusual activities through real-time analysis of large volumes of network traffic. Machine learning models continuously adapt to emerging threats, improving the overall security of Internet services.

Cloud computing platforms integrate AI services that enable developers to build intelligent applications for image recognition, speech processing, natural language understanding, and predictive analytics without requiring specialized hardware. Generative AI has further transformed content creation by producing text, images, audio, videos, and software code, significantly improving productivity across industries.

The combination of AI with the Internet of Things (IoT) enables smart homes, intelligent transportation systems, healthcare monitoring, industrial automation, and smart cities. AI processes data collected from connected devices to make informed decisions and automate complex operations.

Despite its numerous benefits, the AI-driven Internet also presents challenges, including concerns related to privacy, data security, algorithmic bias, misinformation, intellectual property, and ethical use of AI-generated content. Governments and technology organizations worldwide are working to establish regulations and ethical guidelines to ensure responsible AI deployment.

As AI technologies continue to advance, the Internet is expected to become more

intelligent, adaptive, secure, and personalized, supporting innovations such as autonomous systems, intelligent digital assistants, and highly efficient global communication networks.

1.2 Evolution of the World Wide Web (WWW)

The World Wide Web (WWW) is a system of interconnected web pages and digital resources that are accessed over the Internet using web browsers. Although the terms Internet and World Wide Web are often used interchangeably, they are different. The Internet is the global network infrastructure that connects computers worldwide, whereas the World Wide Web is one of the many services that operates on the Internet. The Web uses technologies such as Hypertext Transfer Protocol (HTTP), Hypertext Markup Language (HTML), and Uniform Resource Locator (URL) to enable users to access and navigate information.

The World Wide Web was proposed in 1989 by Tim Berners-Lee while working at CERN. His goal was to create a system that allowed researchers to share and access documents easily through hyperlinks. The first website became available in 1991, introducing the concept of interconnected web pages that could be accessed from anywhere in the world.

The evolution of the World Wide Web can be divided into several generations, each representing significant technological advancements.

The history of the World Wide Web (WWW) is a fascinating journey that transformed from an internal lab project into the global nervous system of modern civilization.

Here is a breakdown of how it all began and evolved:

The Spark: CERN and Tim Berners-Lee (1989)

In 1989, a British scientist named Tim Berners-Lee was working at CERN (the European Organization for Nuclear Research) in Switzerland. Scientists from all over the world needed to share data, but their computers were often incompatible.

To solve this, Berners-Lee wrote a proposal for a "mesh" of interconnected information. By 1990, he had defined the three fundamental technologies that still form the foundation of the Web today:

- **HTML (HyperText Markup Language):** The formatting language for the Web.

- URI/URL (Uniform Resource Identifier/Locator): The unique address used to identify each resource on the Web.
- HTTP (Hypertext Transfer Protocol): The protocol allowing for the retrieval of linked resources across the Web.

Going Public (1991–1993)

- 1991: The very first website went live at CERN (hosted on Berners-Lee's NeXT computer), explaining how the World Wide Web project worked.
- 1993: CERN made a monumental decision: they put the World Wide Web source code into the public domain, ensuring anyone could use and build upon it for free without paying royalties.
- The Mosaic Browser (1993): Developed by Marc Andreessen and Eric Bina at the National Center for Supercomputing Applications (NCSA), Mosaic was the first widely popular web browser. It allowed images to be displayed inline with text, making the Web visual and user-friendly for the general public.

Web 1.0 – The Static Web (1990–2004)

Web 1.0 is considered the first generation of the World Wide Web. During this period, websites primarily consisted of static HTML pages that displayed information without allowing users to interact with the content. Users could read information, but content creation and modification were controlled entirely by website owners.

The main characteristics of Web 1.0 include:

- Static web pages with limited functionality
- One-way communication (read-only)
- Minimal user interaction
- Basic HTML and simple graphics
- Limited multimedia support

Examples of Web 1.0 websites included online directories, digital libraries, educational websites, and company information portals.

Web 2.0 – The Social Web (2004–Present)

Web 2.0 introduced dynamic websites and interactive web applications that allowed users to create, edit, and share content. It transformed users from passive readers into active contributors, giving rise to social networking, blogging, video sharing, and collaborative platforms. Technologies such as AJAX, JavaScript frameworks, cloud computing, and databases enabled real-time interaction and personalized user experiences.

Key characteristics of Web 2.0 include:

- Dynamic and interactive websites
- User-generated content
- Social networking platforms
- Online collaboration and communication
- Rich multimedia support
- Cloud-based applications

Popular examples include social media platforms, video-sharing websites, blogs, online learning platforms, and collaborative tools.

Web 3.0 – The Semantic and Intelligent Web

Web 3.0 represents the next stage in the evolution of the Web by integrating semantic technologies, artificial intelligence, blockchain, and decentralized applications. Unlike previous generations, Web 3.0 focuses on enabling computers to understand the meaning and context of information rather than simply displaying it.

Artificial Intelligence and Machine Learning help deliver personalized search results, intelligent recommendations, and automated decision-making. Blockchain technology supports decentralized applications (dApps), digital ownership, and secure peer-to-peer transactions.

Major characteristics of Web 3.0 include:

- Semantic understanding of information
- AI-powered search and recommendations
- Personalized user experiences

- Decentralized applications (dApps)
- Blockchain-based digital ownership
- Improved privacy and security

Applications of Web 3.0 include cryptocurrency platforms, decentralized finance (DeFi), smart contracts, intelligent virtual assistants, and AI-powered search engines.

Web 4.0 – The Intelligent and Connected Web (Emerging)

Web 4.0 is an emerging concept that aims to create a fully intelligent and interconnected digital ecosystem. It integrates Artificial Intelligence, the Internet of Things (IoT), 5G/6G communication, cloud computing, edge computing, augmented reality (AR), virtual reality (VR), and autonomous systems.

In Web 4.0, devices, services, and users communicate seamlessly with minimal human intervention. Intelligent systems can understand user preferences, predict requirements, and automate complex tasks in real time.

Key features of Web 4.0 include:

- Human-machine collaboration
- AI-driven automation
- Smart connected devices (IoT)
- Real-time personalized services
- Integration of AR, VR, and digital twins
- Autonomous and context-aware applications

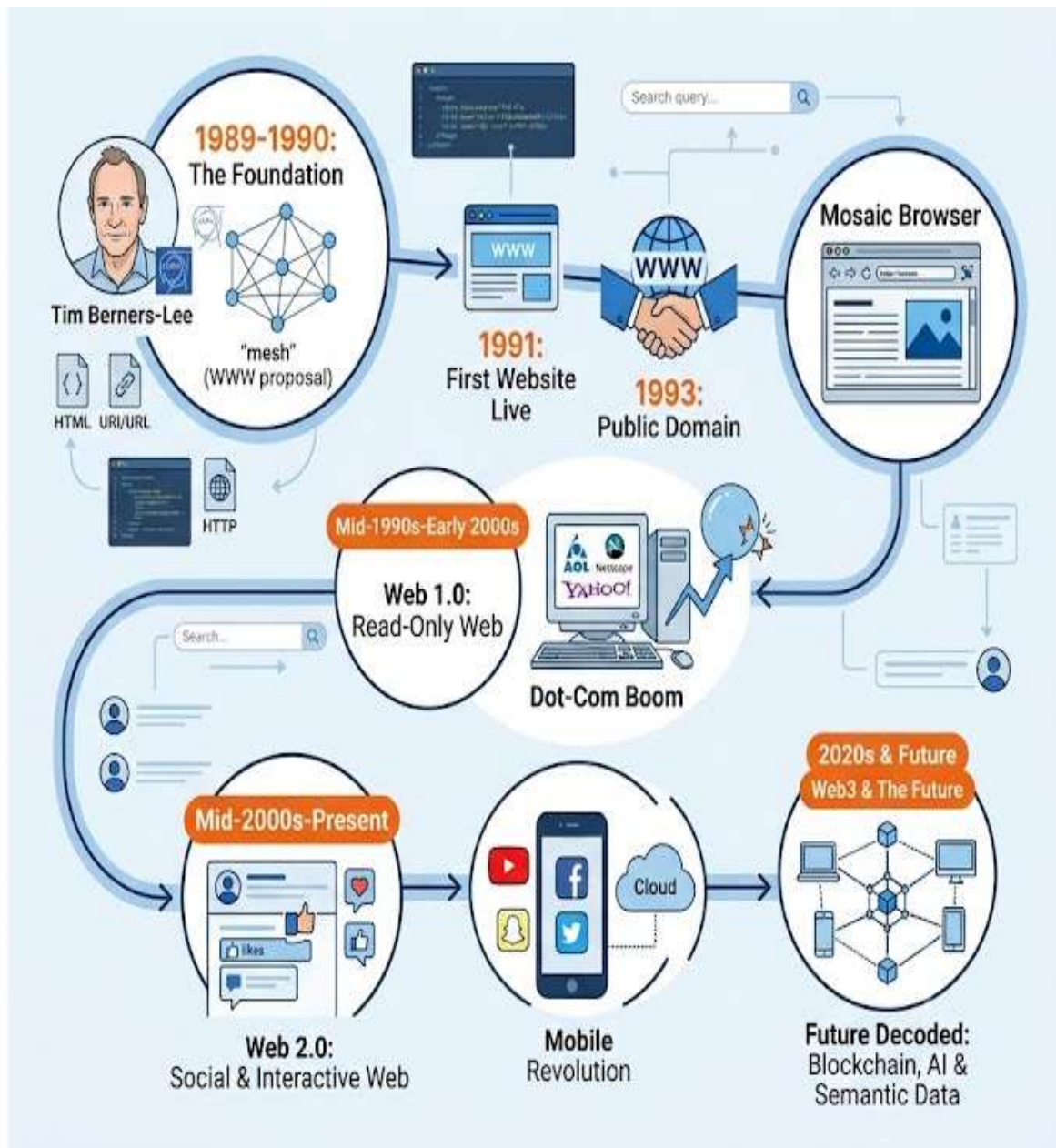
Potential applications include smart cities, autonomous vehicles, intelligent healthcare systems, Industry 4.0, digital education, and immersive online experiences.

Comparison of Web Evolution

Feature	Web 1.0	Web 2.0	Web 3.0	Web 4.0
Nature	Read-only	Read-Write	Read-Write-Execute	Intelligent and Autonomous
Content	Static	Dynamic	Semantic and Personalized	Intelligent and Predictive
User Role	Consumer	Contributor	Creator and Owner	Human-AI Collaboration
Technologies	HTML	AJAX, JavaScript, Cloud	AI, Blockchain, Semantic Web	AI, IoT, Edge Computing, AR/VR
Interaction	Limited	High	Intelligent	Context-aware and Autonomous
Examples	Company websites	Social media, Blogs	dApps, AI assistants	Smart cities, Autonomous systems

Evolution of WWW

Year/Period	Milestone	Key Development
1989	Web proposed	Tim Berners-Lee proposes the WWW at CERN
1990	First browser	First web browser and web server developed
1991	First website	HTML, HTTP, and URL introduced
1993	Mosaic browser	Public adoption of the Web increases
1994–1999	Commercial Web	Search engines, e-commerce, and dynamic websites emerge
2004–Present	Web 2.0	Interactive websites, social media, blogs, cloud applications
2020–Present	Web 3.0	AI, Semantic Web, Blockchain, decentralized applications
2025+	Web 4.0	Intelligent, autonomous, AI-driven, and IoT-enabled Web



The World Wide Web has evolved from a collection of static web pages into an intelligent, interactive, and decentralized digital ecosystem. Each generation has introduced new technologies that have improved communication, collaboration, information sharing, and online services. Today, the convergence of Artificial Intelligence, cloud computing, blockchain, and IoT is shaping the future of the Web, making it more personalized, secure, and intelligent while enabling innovative applications across education, healthcare, business, and industry.

1.3 Web Browsers and Web Servers

The **World Wide Web (WWW)** operates through the interaction between **web browsers** and **web servers**. A web browser is a software application that enables users to access and display web pages, while a web server is a computer system or software that stores, processes, and delivers web content to users over the Internet. Together, they form the foundation of web communication using the **Hypertext Transfer Protocol (HTTP)** and **Hypertext Transfer Protocol Secure (HTTPS)**.

When a user enters a website address (URL) into a web browser, the browser sends a request to the appropriate web server. The web server processes the request and returns the requested webpage, which the browser interprets and displays using HTML, CSS, JavaScript, images, and multimedia content.

1.3.1 Web Browser

A **web browser** is a client-side application used to access, retrieve, and display information available on the World Wide Web. It provides a graphical user interface (GUI) that enables users to navigate between websites through hyperlinks and URLs.

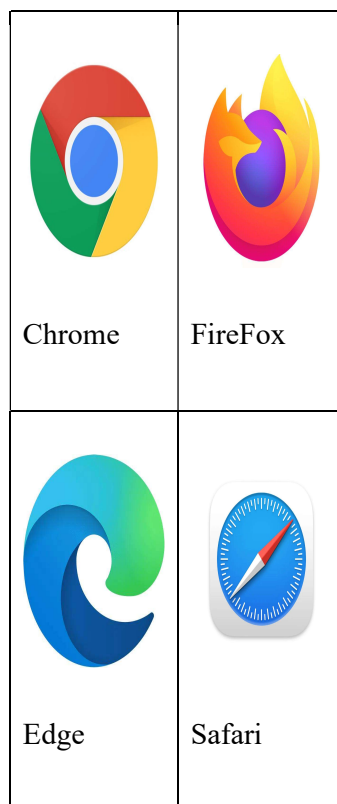
A web browser communicates with web servers using HTTP or HTTPS protocols. It downloads web resources such as HTML documents, CSS stylesheets, JavaScript files, images, videos, and other multimedia content, rendering them into a user-friendly webpage.

Functions of a Web Browser

- Retrieves web pages from web servers.
- Displays text, images, videos, and animations.
- Executes JavaScript to support interactive web applications.
- Stores browsing history and bookmarks.
- Supports secure browsing through HTTPS encryption.
- Manages cookies and cached files for faster page loading.
- Provides extensions and developer tools.

Popular Web Browsers

- **Google Chrome** – Developed by Google; the most widely used browser known for speed and extensive extension support.
- **Mozilla Firefox** – Developed by Mozilla Foundation; emphasizes privacy and open-source development.
- **Microsoft Edge** – Developed by Microsoft; built on the Chromium engine and integrated with Windows.
- **Safari** – Developed by Apple; optimized for macOS and iOS devices.
- **Opera** – Offers built-in VPN, ad blocking, and battery-saving features.



1.3.2 Web Server

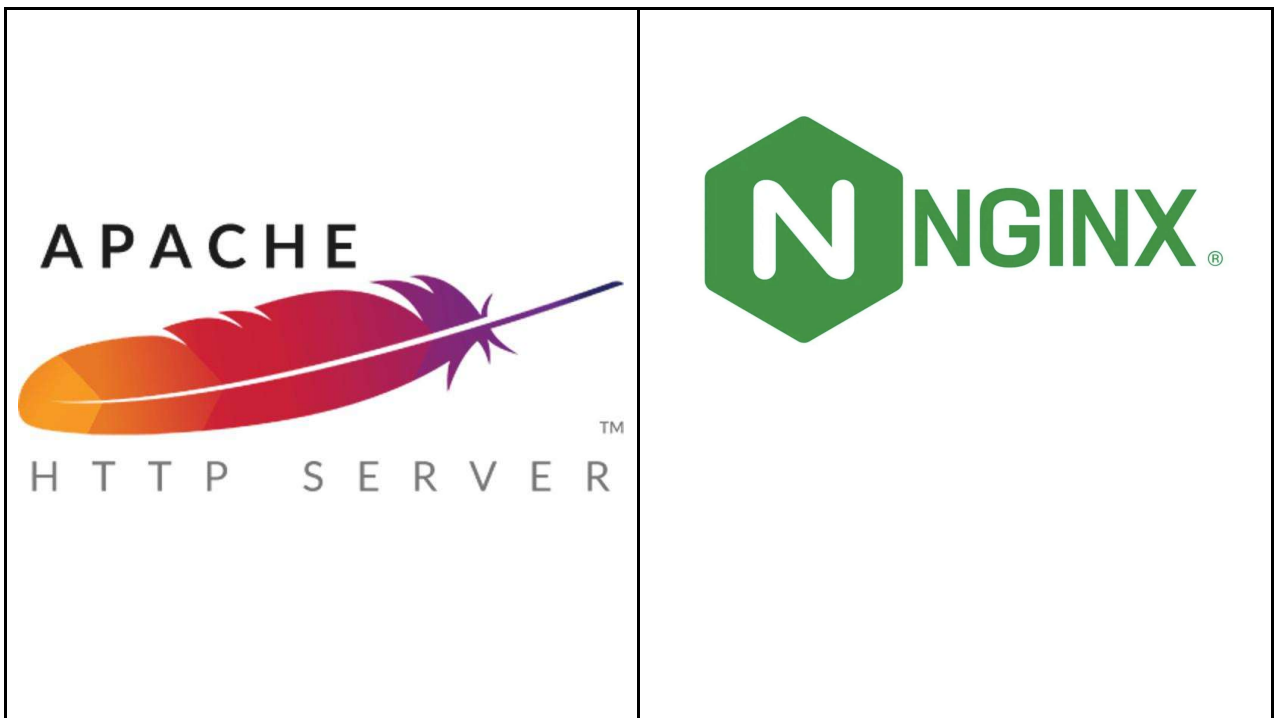
A **web server** is a computer or software application that hosts websites and delivers web pages to users upon request. It stores website files such as HTML pages, CSS files, JavaScript programs, images, videos, and databases.

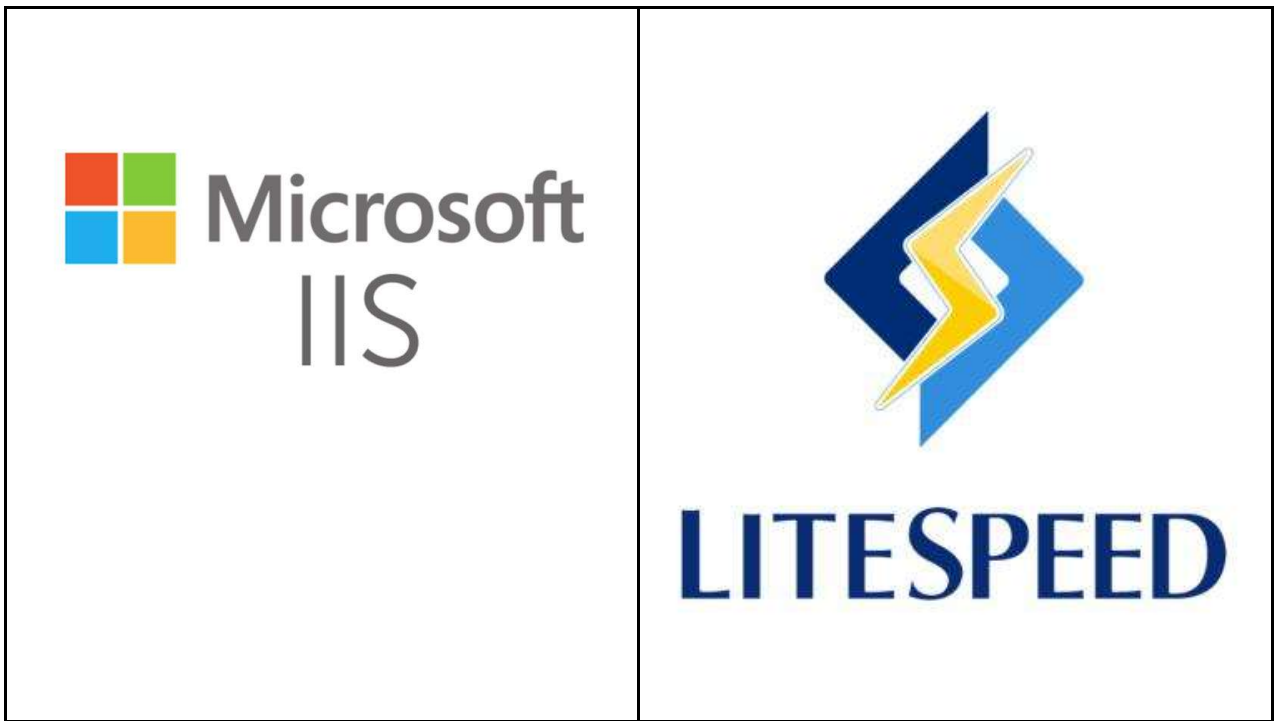
When a browser sends an HTTP request, the web server locates the requested resource, processes the request, and sends the appropriate response back to the browser.

Functions of a Web Server

- Hosts websites and web applications.
- Processes client requests.
- Delivers web pages and multimedia files.
- Supports dynamic web applications using server-side scripting.
- Manages user authentication and sessions.
- Provides secure communication through HTTPS.
- Handles multiple client requests simultaneously.

Popular Web Servers





- **Apache HTTP Server** – One of the most popular open-source web servers.
- **Nginx** – Known for high performance, scalability, and efficient load balancing.
- **Microsoft Internet Information Services** – Commonly used for ASP.NET applications on Windows servers.
- **LiteSpeed Web Server** – High-performance commercial web server with built-in caching.

Working of a Web Browser and Web Server

The communication between a browser and a web server follows these steps:

1. The user enters a website URL in the web browser.
2. The browser queries the Domain Name System (DNS) to obtain the server's IP address.
3. The browser sends an HTTP or HTTPS request to the web server.
4. The web server processes the request and retrieves the required webpage or data.
5. The server sends an HTTP response containing the requested resources.
6. The browser renders the HTML, CSS, JavaScript, images, and multimedia into a webpage for the user.

Difference Between Web Browser and Web Server

Feature	Web Browser	Web Server
Definition	Software used to access websites	Software or computer that hosts websites
Purpose	Displays web pages to users	Stores and delivers web pages
Role	Client	Server
Communication	Sends HTTP/HTTPS requests	Responds to HTTP/HTTPS requests
Installation	Installed on the user's device	Installed on a server machine
Examples	Chrome, Firefox, Edge, Safari	Apache, Nginx, IIS, LiteSpeed

A **web browser** acts as the client that requests web pages, while a **web server** stores and delivers those pages over the Internet. They communicate using HTTP or HTTPS protocols to provide users with seamless access to websites and web applications. Understanding the roles of browsers and servers is fundamental to learning web technologies, as every web application relies on this client–server architecture.

1.4 Client–Server Architecture

The **Client–Server Architecture** is a computing model in which multiple client devices request services or resources from a centralized server over a network. It forms the foundation of most Internet-based applications, including websites, email services, online banking, cloud computing, e-commerce, and social media platforms.

In this architecture, the **client** is responsible for sending requests, while the **server** processes those requests and provides the required resources or services. Communication between the client and the server typically takes place using standard protocols such as **HTTP**, **HTTPS**, **FTP**, and **SMTP** over the Internet.

The client–server model enables centralized data management, efficient resource sharing, enhanced security, and easier maintenance. Since servers are capable of handling multiple client requests simultaneously, this architecture supports scalability and reliability in modern web applications.

1.4.1 Components of Client–Server Architecture

A client–server system consists of the following major components:

1. Client

A **client** is a computer, smartphone, tablet, or software application that requests services or resources from a server.

Functions of a Client

- Sends requests to the server.
- Displays the received information to the user.
- Provides a user interface for interaction.
- Processes user inputs before sending requests.

Examples

- Web browsers (Chrome, Firefox, Edge)
- Mobile applications
- Desktop applications
- Smart TV applications

2. Server

A **server** is a powerful computer or software system that stores data, hosts applications, and provides services to multiple clients simultaneously.

Functions of a Server

- Receives client requests.
- Processes requests using business logic.
- Retrieves information from databases.
- Sends responses back to clients.
- Manages security and user authentication.

Examples

- Web Server
- Database Server
- File Server
- Mail Server
- Cloud Server

3. Communication Network

The communication network connects clients and servers through wired or wireless technologies.

Examples include:

- Internet
- Local Area Network (LAN)
- Wide Area Network (WAN)
- Wi-Fi
- Mobile Networks (4G/5G)

4. Database

The database stores application data that can be retrieved or updated by the server based on client requests.

Examples include:

- MySQL

- PostgreSQL
- Oracle Database
- Microsoft SQL Server
- MongoDB

1.4.2 Working of Client–Server Architecture

The client–server architecture operates through the following sequence:

1. The user enters a request using a client application.
2. The client sends the request to the server through the network.
3. The server receives and processes the request.
4. If required, the server retrieves or updates information in the database.
5. The server generates a response.
6. The response is transmitted back to the client.
7. The client displays the requested information to the user.

1.4.3 Types of Client–Server Architecture

1. Two-Tier Architecture

In a two-tier architecture, the client communicates directly with the server.

Characteristics

- Simple design
- Faster communication
- Suitable for small applications
- Easy to develop and maintain

Example

A desktop application directly accessing a database server.

2. Three-Tier Architecture

Three-tier architecture introduces an **application server** between the client and the database server.

The three layers are:

- **Presentation Layer** – User interface
- **Application Layer** – Business logic
- **Data Layer** – Database management

Advantages

- Better security
- Easier maintenance
- High scalability
- Improved performance

Example

Online banking systems, e-commerce websites, and university management systems.

3. Multi-Tier (N-Tier) Architecture

Large enterprise applications use multiple layers such as web servers, application servers, API servers, cache servers, and database servers.

Advantages

- High availability
- Load balancing
- Fault tolerance
- Supports cloud computing
- Easier scalability

Examples include cloud-based applications, enterprise software, and large-scale e-commerce platforms.

1.4.4 Advantages of Client–Server Architecture

- Centralized data management.
- Easier maintenance and software updates.
- Improved security through centralized authentication.
- Efficient sharing of resources.

- Supports multiple users simultaneously.
- Scalable for growing applications.
- Reliable backup and recovery mechanisms.
- Better performance through dedicated servers.

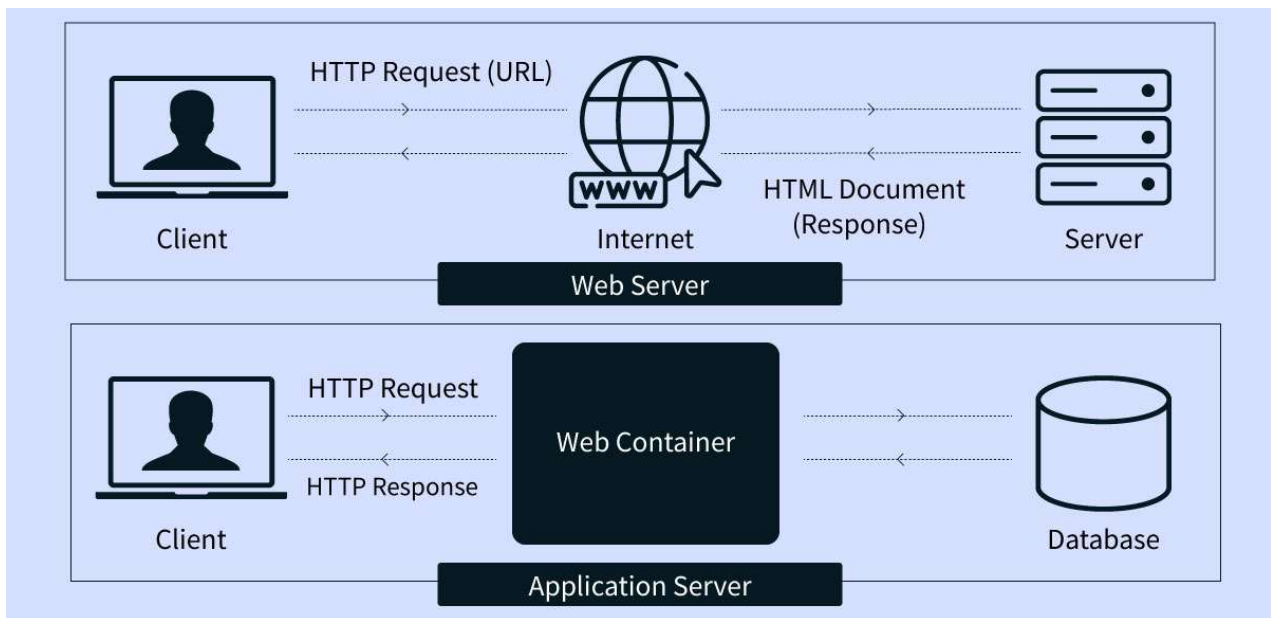
1.4.5 Disadvantages of Client–Server Architecture

- Server failure may affect all connected clients.
- High-performance servers can be expensive.
- Network dependency may affect application performance.
- Maintenance requires skilled administrators.
- Heavy traffic may reduce server performance.

1.4.6 Applications of Client–Server Architecture

Client–server architecture is widely used in numerous real-world applications, including:

- Web applications
- E-commerce websites
- Online banking systems
- Email services
- Cloud computing platforms
- Social networking platforms
- Video streaming services
- Online gaming
- Hospital management systems
- University management systems



1.5 HTTP and HTTPS Protocols

The **Hypertext Transfer Protocol (HTTP)** and **Hypertext Transfer Protocol Secure (HTTPS)** are application-layer communication protocols used for transmitting information between web browsers (clients) and web servers over the Internet. They define the rules and standards for requesting and delivering web pages, images, videos, and other web resources.

HTTP is the foundation of data communication on the World Wide Web. However, since HTTP transmits data in plain text, it is vulnerable to eavesdropping, tampering, and cyberattacks. To overcome these security issues, HTTPS was introduced. HTTPS combines HTTP with **Transport Layer Security (TLS)** or its predecessor **Secure Sockets Layer (SSL)** to provide encrypted and secure communication between clients and servers.

Today, almost all modern websites use HTTPS to protect sensitive information such as passwords, banking details, personal information, and online transactions.

1.5.1 Hypertext Transfer Protocol (HTTP)

HTTP (Hypertext Transfer Protocol) is a stateless application-layer protocol used for transferring web resources between a client and a server. It follows a request–response model, where the client sends a request, and the server returns the appropriate response.

HTTP is a stateless protocol, meaning each request is treated independently without retaining information about previous requests. To maintain user sessions, technologies such as cookies

and session management are used.

Features of HTTP

- Client–server communication model
- Stateless protocol
- Request–response communication
- Supports multimedia content
- Platform independent
- Easy to implement
- Fast communication for non-sensitive data

HTTP Communication Process

1. User enters a website URL in the browser.
2. Browser sends an HTTP request to the web server.
3. Server processes the request.
4. Server sends an HTTP response.
5. Browser displays the webpage.

1.5.2 Hypertext Transfer Protocol Secure (HTTPS)

HTTPS (Hypertext Transfer Protocol Secure) is the secure version of HTTP. It encrypts communication using **TLS (Transport Layer Security)**, ensuring confidentiality, integrity, and authentication of data exchanged between clients and servers.

HTTPS protects users from cyber threats such as data interception, identity theft, and man-in-the-middle attacks. Modern browsers indicate secure websites using a padlock icon and the prefix **https://**.

Features of HTTPS

- Encrypts transmitted data
- Provides authentication using digital certificates
- Ensures data integrity
- Protects user privacy

- Builds user trust
- Required for secure online transactions

1.5.3 HTTP Request Methods

HTTP defines several request methods that specify the action to be performed on a resource.

Method	Description
GET	Retrieves data from the server
POST	Sends data to the server
PUT	Updates an existing resource
DELETE	Removes a resource
PATCH	Partially updates a resource
HEAD	Retrieves only response headers
OPTIONS	Returns supported communication methods

1.5.4 Structure of HTTP Request

An HTTP request consists of:

- **Request Line** – Contains the HTTP method, resource path, and protocol version.
- **Headers** – Provide additional information such as browser type, content type, and authentication.
- **Body** – Contains data sent to the server (mainly in POST and PUT requests).

Example HTTP Request

GET /index.html HTTP/1.1

Host: www.example.com

User-Agent: Mozilla/5.0

Accept: text/html

1.5.5 Structure of HTTP Response

The server sends an HTTP response containing:

- Status Line
- Response Headers
- Response Body

Example HTTP Response

HTTP/1.1 200 OK

Content-Type: text/html

Content-Length: 2048

<html>

...

</html>

1.5.6 Common HTTP Status Codes

HTTP status codes indicate the outcome of a client's request.

Status Code	Meaning
100 Continue	Request received; continue processing
200 OK	Request completed successfully
201 Created	Resource successfully created
301 Moved Permanently	Resource has a new URL
302 Found	Temporary redirection
304 Not Modified	Cached version can be used
400 Bad Request	Invalid client request
401 Unauthorized	Authentication required
403 Forbidden	Access denied
404 Not Found	Requested resource not found
405 Method Not Allowed	HTTP method not supported
500 Internal Server Error	Server-side error
502 Bad Gateway	Invalid response from another server
503 Service Unavailable	Server temporarily unavailable

1.5.7 Difference Between HTTP and HTTPS

Feature	HTTP	HTTPS
Full Form	Hypertext Transfer Protocol	Hypertext Transfer Protocol Secure
Security	Not secure	Secure
Encryption	No	Yes (TLS/SSL)
Default Port	80	443
URL Prefix	http://	https://
Digital Certificate	Not required	Required
Data Protection	Plain text	Encrypted
Suitable For	Public information	Sensitive information and online transactions

1.5.8 Advantages of HTTPS

- Encrypts data during transmission.
- Prevents eavesdropping and data theft.
- Authenticates the identity of websites.
- Protects against man-in-the-middle attacks.
- Improves user trust and confidence.
- Enhances search engine ranking.
- Required for secure online payments and banking.
- Supports modern web technologies and browser security features.

1.5.9 Applications of HTTP and HTTPS

Applications of HTTP

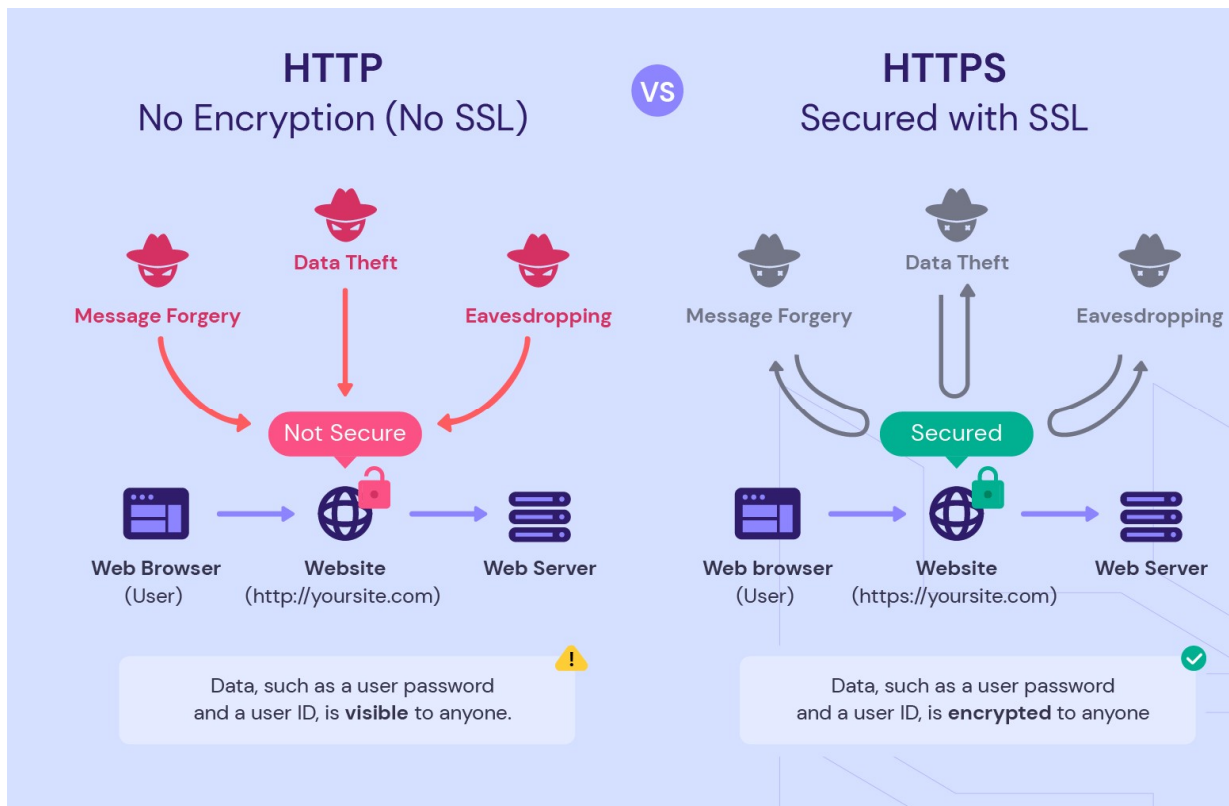
- Educational websites
- Public information portals
- Internal testing environments
- Static web content

Applications of HTTPS

- Internet banking
- E-commerce websites
- Online payment gateways
- Social media platforms
- Email services
- Government portals
- Healthcare applications
- Cloud-based services

1.5.10 HTTP vs HTTPS Communication Flow

HTTP and HTTPS are essential protocols that enable communication between web browsers and web servers. HTTP provides a simple request–response mechanism for accessing web resources, whereas HTTPS enhances this communication by incorporating encryption, authentication, and data integrity through TLS. As cybersecurity threats continue to grow, HTTPS has become the standard protocol for secure web communication, ensuring safe browsing, online transactions, and protection of sensitive user information.



1.6 URL, URI and Domain Name System (DNS) Basics

The Internet consists of millions of interconnected computers and websites that exchange information continuously. To access a particular resource on the Internet, users require a unique identifier and a mechanism to locate the corresponding server. **Uniform Resource Identifiers (URIs)** and **Uniform Resource Locators (URLs)** provide standardized methods for identifying and locating resources, while the **Domain Name System (DNS)** translates human-readable domain names into machine-readable Internet Protocol (IP) addresses.

Together, URLs, URIs, and DNS form the backbone of web navigation, enabling users to access websites and online services without needing to remember complex numerical IP addresses.

1.6.1 Uniform Resource Identifier (URI)

A **Uniform Resource Identifier (URI)** is a string of characters that uniquely identifies a resource on the Internet. The resource may be a webpage, image, video, document, or any other digital object.

A URI identifies a resource either by its **location**, **name**, or **both**. It serves as the general framework for identifying resources on the Web.

General Syntax of URI

scheme : identifier

Examples

https://www.example.com/index.html

mailto:student@example.com

ftp://ftp.example.com/files/report.pdf

urn:isbn:9780134685991

Characteristics of URI

- Uniquely identifies Internet resources.
- Supports multiple schemes such as HTTP, HTTPS, FTP, Mailto, and URN.
- May specify the resource location or simply identify the resource.
- Provides a standardized method for resource identification.

1.6.2 Uniform Resource Locator (URL)

A **Uniform Resource Locator (URL)** is a specific type of URI that identifies the location of a resource and specifies the protocol used to access it.

A URL tells the browser:

- Which protocol to use.
- Which server to contact.
- Which resource to retrieve.

General Structure of URL

protocol://domain-name:port/path?query#fragment

Example

<https://www.example.com:443/products/laptop.html?id=101#reviews>

Components of a URL

Component	Description	Example
Protocol (Scheme)	Communication protocol	https
Domain Name	Website name	www.example.com
Port Number	Communication port	443
Path	Location of resource	/products/laptop.html
Query String	Parameters sent to server	id=101
Fragment	Section within page	reviews

Characteristics of URL

- Specifies the exact location of a resource.
- Uses standard Internet protocols.
- Human-readable format.
- Enables browsers to retrieve web resources.

1.6.3 Difference Between URI and URL

Feature	URI	URL
Full Form	Uniform Resource Identifier	Uniform Resource Locator
Purpose	Identifies a resource	Identifies and locates a resource
Includes Location	Not always	Always
Includes Protocol	Optional	Yes
Scope	Broader concept	Subset of URI
Example	urn:isbn:9780134685991	https://www.example.com

Note: Every **URL** is a **URI**, but **not every URI is a URL**.

1.6.4 Domain Name System (DNS)

The **Domain Name System (DNS)** is often referred to as the "**phonebook of the Internet**." It is a hierarchical and distributed naming system that translates domain names (such as **www.google.com**) into numerical IP addresses (such as **142.250.182.14**) that computers use for communication.

Without DNS, users would need to remember the IP address of every website they visit. DNS makes Internet navigation easier by allowing users to access websites using meaningful names.

Functions of DNS

- Converts domain names into IP addresses.
- Enables easy access to websites.
- Maintains distributed domain databases.
- Supports load balancing.
- Improves Internet scalability and reliability.
- Reduces the need to remember numerical IP addresses.

1.6.5 Components of DNS

1. Domain Name

A human-readable name assigned to a website.

Example:

www.example.com

2. DNS Resolver

Receives DNS queries from client devices and searches for the corresponding IP address.

3. Root Name Server

The highest level of the DNS hierarchy. It directs queries to the appropriate Top-Level Domain (TLD) servers.

4. Top-Level Domain (TLD) Server

Maintains information about domain extensions.

Examples:

- .com
- .org
- .edu
- .gov

- .in

5. Authoritative Name Server

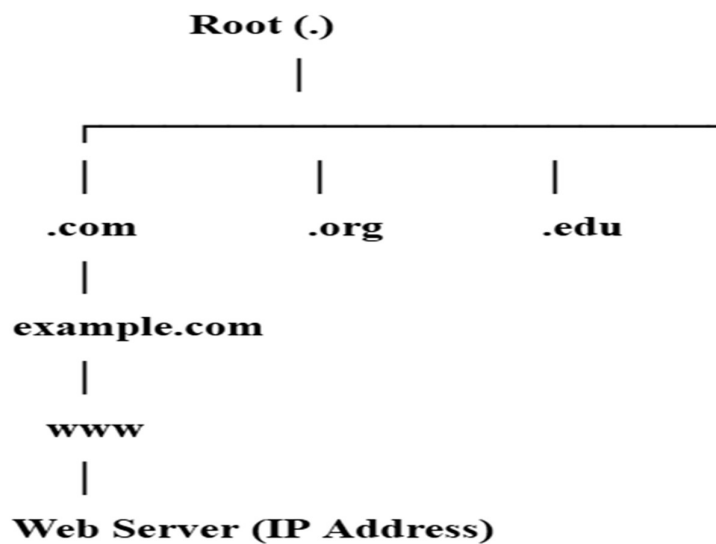
Stores the actual DNS records for a domain and returns the correct IP address.

1.6.6 DNS Resolution Process

The DNS lookup process occurs as follows:

1. The user enters a website URL into the browser.
2. The browser checks its local DNS cache.
3. If unavailable, the request is sent to the DNS resolver.
4. The resolver contacts the Root DNS server.
5. The Root server directs the query to the appropriate TLD server.
6. The TLD server identifies the Authoritative Name Server.
7. The Authoritative server returns the website's IP address.
8. The browser connects to the web server using the IP address.
9. The requested webpage is displayed

1.6.7 DNS Hierarchy



1.6.8 Common DNS Record Types

Record	Purpose
A	Maps domain name to IPv4 address
AAAA	Maps domain name to IPv6 address
CNAME	Alias for another domain
MX	Mail server information
NS	Name server information
TXT	Stores verification and security information
PTR	Reverse DNS lookup

1.6.9 Advantages of DNS

- Easy-to-remember website names.
- Faster website access through caching.
- Distributed architecture improves reliability.
- Supports load balancing.
- Simplifies Internet navigation.
- Scalable for billions of Internet devices.
- Enables email routing and other Internet services.

1.6.10 Difference Between URL, URI, and DNS

Feature	URI	URL	DNS
Purpose	Identifies a resource	Locates a resource	Resolves domain names
Human Readable	Yes	Yes	Yes
Uses Protocol	Optional	Yes	No
Provides Location	Not always	Yes	No
Converts Domain to IP	No	No	Yes
Example	urn:isbn:9780134685991	https://www.example.com/index.html	www.example.com → 93.184.216.34

1.6.11 Real-World Example

Suppose a user enters the following URL:

<https://www.example.com/about.html>

The sequence of events is:

1. The browser recognizes **https** as the communication protocol.
2. The browser extracts the domain name www.example.com.
3. DNS translates the domain name into an IP address.
4. The browser establishes a secure connection with the web server.
5. The server sends the requested webpage.
6. The browser renders and displays the webpage to the user.

Uniform Resource Identifiers (URIs), Uniform Resource Locators (URLs), and the Domain Name System (DNS) are fundamental components of web communication. A **URI** uniquely identifies a resource, while a **URL** specifies both the resource's identity and its location. The **Domain Name System (DNS)** translates human-readable domain names into IP addresses, allowing browsers to locate web servers efficiently. Together, these technologies provide a seamless and user-friendly mechanism for accessing websites and Internet services, making them indispensable to the functioning of the World Wide Web.

1.7 HTML Basics and Document Structure

HyperText Markup Language (HTML) is the standard markup language used to create and structure web pages on the World Wide Web. It defines the content and layout of a webpage using a collection of elements called **HTML tags**. HTML is not a programming language; instead, it is a markup language that describes the structure and organization of web content.

HTML works together with **Cascading Style Sheets (CSS)** and **JavaScript** to build modern web applications. HTML provides the structure of a webpage, CSS controls its presentation and appearance, and JavaScript adds interactivity and dynamic behavior.

Every webpage on the Internet is fundamentally based on HTML. Web browsers interpret HTML code and render it into a visual webpage that users can view and interact with.

1.7.1 Features of HTML

HTML offers several features that make it the foundation of web development:

- Simple and easy to learn.
- Platform independent.
- Supports hyperlinks between web pages.
- Integrates multimedia such as images, audio, and videos.
- Supports tables, lists, forms, and frames.
- Compatible with all modern web browsers.
- Supports semantic elements for better accessibility and SEO.
- Can be combined with CSS and JavaScript to create interactive websites.

1.7.2 Basic Structure of an HTML Document

Every HTML document follows a standard structure consisting of the document declaration, head section, and body section.

HTML Document Structure

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
    <title>My First Web Page</title>
```

```
</head>
```

```
<body>
```

```
    <h1>Welcome to HTML</h1>
```

```
    <p>This is my first HTML webpage.</p>
```

```
</body>
```

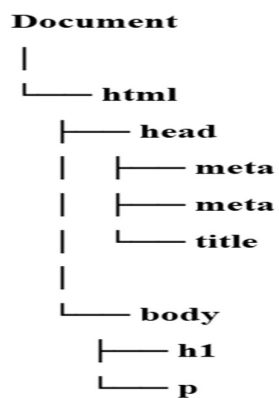
```
</html>
```

1.7.3 Explanation of HTML Document Structure

HTML Element	Purpose
<code><!DOCTYPE html></code>	Declares the document as an HTML5 document.
<code><html></code>	Root element containing the entire webpage.
<code><head></code>	Stores metadata such as title, CSS links, and scripts.
<code><meta></code>	Specifies character encoding and viewport settings.
<code><title></code>	Displays the webpage title in the browser tab.
<code><body></code>	Contains all visible webpage content.

1.7.4 HTML Document Tree

The HTML document is organized as a hierarchical tree known as the **Document Object Model (DOM)**.



Each HTML element becomes a **node** in the DOM tree, allowing JavaScript to access and manipulate webpage content dynamically.

1.7.5 HTML Tags

HTML uses **tags** enclosed within angle brackets (`<>`) to define webpage elements.

Syntax

`<tagname>Content</tagname>`

Example:

`<h1>Welcome</h1>`

`<p>This is HTML.</p>`

Some HTML tags are **paired tags**, while others are **empty tags**.

Paired Tags

Contain both opening and closing tags.

Example:

`<p>This is a paragraph.</p>`

Empty Tags

Do not require a closing tag.

Examples:

`<br>`

`<hr>`

``

1.7.6 HTML Elements

An **HTML element** consists of:

- Opening tag
- Content
- Closing tag

Example:

```
<p>Learning HTML is easy.</p>
```

Here,

- Opening Tag → `<p>`
- Content → Learning HTML is easy.
- Closing Tag → `</p>`

1.7.7 HTML Attributes

Attributes provide additional information about HTML elements.

Syntax

```
<tag attribute="value">
```

Example:

```

```

Common attributes include:

Attribute	Purpose
id	Unique identifier
class	Defines CSS class
href	Specifies hyperlink
src	Specifies image or media source
alt	Alternative text for images
title	Tooltip information
style	Inline CSS styling

1.7.8 Basic HTML Tags

Heading Tags

HTML provides six levels of headings.

`<h1>Main Heading</h1>`

`<h2>Sub Heading</h2>`

`<h3>Section Heading</h3>`

`<h4>Topic Heading</h4>`

`<h5>Minor Heading</h5>`

`<h6>Smallest Heading</h6>`

Paragraph Tag

`<p>This is a paragraph.</p>`

Line Break

Line One

Line Two

Horizontal Rule

<hr>

Bold, Italic and Underline

Bold Text

<i>Italic Text</i>

<u>Underlined Text</u>

1.7.9 HTML Comments

Comments help developers document the source code.

<!-- This is an HTML comment -->

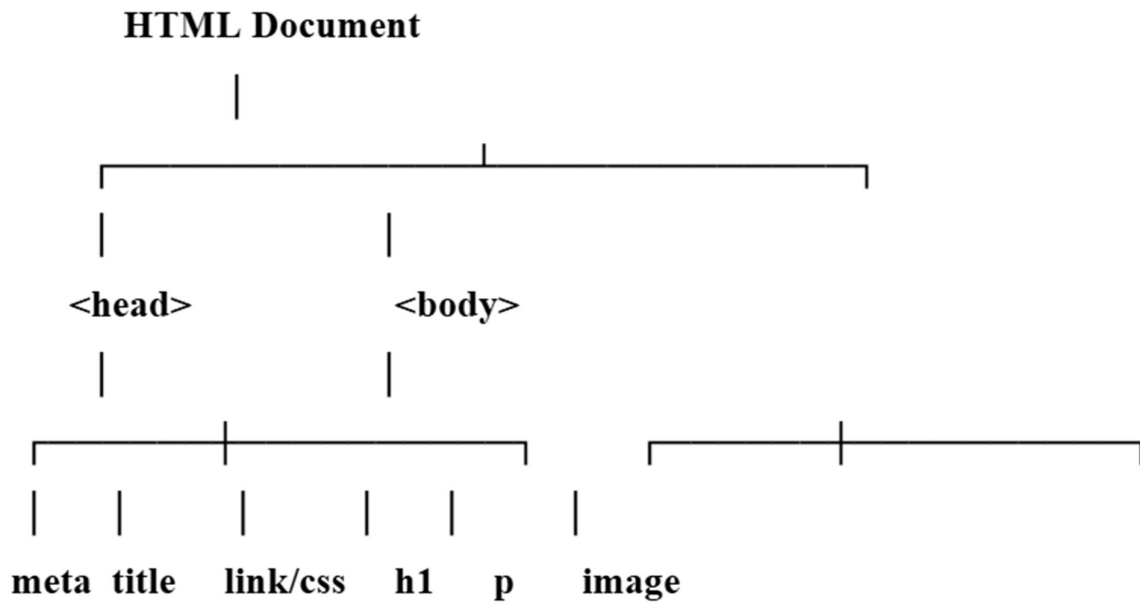
Comments are ignored by the browser and are not displayed on the webpage.

1.7.10 HTML Document Workflow

The process of creating and displaying an HTML webpage involves the following steps:

1. Create an HTML file using a text editor or IDE.
2. Save the file with the `.html` extension.
3. Open the file in a web browser.
4. The browser parses the HTML code.
5. The browser constructs the Document Object Model (DOM).
6. The webpage is rendered and displayed to the user.

1.7.11 HTML Document Structure Diagram



1.7.12 Advantages of HTML

- Easy to learn and use.
- Open standard supported by all browsers.
- Platform independent.
- Supports multimedia integration.
- Works seamlessly with CSS and JavaScript.
- Provides semantic structure for web content.
- Improves accessibility and search engine optimization.
- Lightweight and fast to load.

1.7.13 Limitations of HTML

- Cannot perform complex programming logic.
- Cannot create dynamic web pages by itself.
- Requires CSS for styling.
- Requires JavaScript for interactivity.
- Does not directly connect to databases.

Summary

HTML (HyperText Markup Language) is the standard markup language used to create and structure web pages. It provides a well-defined document structure consisting of the `<!DOCTYPE>`, `<html>`, `<head>`, and `<body>` elements. HTML uses tags, elements, and attributes to organize content such as headings, paragraphs, images, links, and multimedia. Although HTML defines the structure of a webpage, it is typically used alongside CSS for presentation and JavaScript for dynamic functionality. A solid understanding of HTML basics and document structure forms the foundation for learning advanced web technologies and web application development.

1.8 Creating a Simple Web Page

Exercise

1. **Create your profile page**
2. **Create a login page**
3. **Create an information page about your native**
4. **Create a ECommerce website using FlexBox and Grid**
5. Design static web pages using HTML and CSS.
6. **Develop responsive layouts using Flexbox/Grid.**

Develop responsive layouts using Flexbox/Grid.

What is a Responsive Layout?

A responsive layout is a web page design that automatically adjusts its structure, sizing, and arrangement based on the screen size or device being used (mobile, tablet, laptop, desktop) — without requiring separate versions of the website for each device.

Instead of a fixed, one-size-fits-all layout, responsive design uses flexible grids, flexible images, and CSS techniques (like Flexbox, Grid, and media queries) so the content reflows to fit any screen.

Why it matters

- A user on a phone sees a single-column, stacked layout.
- The same site on a laptop shows a multi-column layout — without you building two different websites.

Flexbox Example (1D layout — row or column)

html

```
<div class="flex-container">  
  <div class="box">1</div>  
  <div class="box">2</div>  
  <div class="box">3</div>  
</div>
```

css

```
.flex-container {  
  display: flex;  
  gap: 10px;  
}  
  
.box {  
  flex: 1;  
  background: lightblue;  
  padding: 20px;  
  text-align: center;  
}
```

What happens: The three boxes sit in a row, each taking equal width (**flex: 1**). Flexbox is best for one-dimensional layouts — a navbar, a row of buttons, or cards in a line.

Grid Example (2D layout — rows AND columns)

html

```
<div class="grid-container">  
  <div class="box">1</div>  
  <div class="box">2</div>  
  <div class="box">3</div>
```

```

<div class="box">4</div>
</div>

css

.grid-container {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 10px;
}

.box {
  background: lightgreen;
  padding: 20px;
  text-align: center;
}

```

What happens: The four boxes arrange into a 2x2 grid (2 columns, auto rows). Grid is best for two-dimensional layouts — a full page structure with header, sidebar, content, and footer.

Key difference to emphasize to students

	Flexbox	Grid
Dimension	1D (row or column)	2D (rows and columns)
Best for	Navbars, card rows, aligning items	Page layouts, image galleries, dashboards
Main property	display: flex	display: grid